

Lab: Implementing Data Flow in an SSIS Package

Scenario

In this lab, you will focus on the extraction of customer and sales order data from the **InternetSales** database used by the company's e-commerce site, which you must load into the **Staging** database. This database contains customer data (in a table named **Customers**), and sales order data (in tables named **SalesOrderHeader** and **SalesOrderDetail**). You will extract sales order data at the line item level of granularity. The total sales amount for each sales order line item is then calculated by multiplying the unit price of the product purchased by the quantity ordered. Additionally, the sales order data includes only the ID of the product purchased, so your data flow must look up the details of each product in a separate **Products** database.

Objectives

After completing this lab, you will be able to:

- Extract and profile source data.
- Implement a data flow.
- Use transformation components in a data flow.

Estimated Time: 60 minutes

Virtual machine: **20767C-MIA-SQL**

User name: **ADVENTUREWORKS\Student**

Password: **Pa55w.rd**

Exercise 1: Exploring Source Data

Scenario

You have designed a data warehouse schema for Adventure Works Cycles. You must now design an ETL process to populate it with data from various source systems. Before creating the ETL solution, you have decided to examine the source data so you can understand it better.

The main tasks for this exercise are as follows:

1. Prepare the Lab Environment
2. Extract and View Sample Source Data
3. Profile Source Data

► Task 1: Prepare the Lab Environment

1. Ensure that the **20767C-MIA-DC** and **20767C-MIA-SQL** virtual machines are both running, and then log on to **20767C-MIA-SQL** as **ADVENTUREWORKS\Student** with the password **Pa55w.rd**
2. In the **20767C-MIA-SQL** virtual machine, run **Setup.cmd** in the **D:\Labfiles\Lab06\Starter** folder as Administrator.

► Task 2: Extract and View Sample Source Data

1. Use the SQL Server Import and Export Data Wizard to extract a sample of customer data from the **InternetSales** database on the **localhost** instance of SQL Server to a comma-delimited flat file.
 - Your sample should consist of the first 1,000 records in the **Customers** table.
 - You should use a text qualifier because some string values in the table may contain commas.

2. After you have extracted the sample data, use Notepad to view it.

Note: You may observe some anomalies in the data, such as invalid gender codes and multiple values for the same country or region. The purpose of examining the source data is to identify as many of these problems as possible, so that you can resolve them in the development of the ETL solution. You will address the problems in this data in later labs.

► Task 3: Profile Source Data

1. Create an Integration Services project named **Explore Internet Sales** in the **D:\Labfiles\Lab06\Starter** folder.
2. Add an ADO.NET connection manager that uses Windows authentication to connect to the **InternetSales** database on the **localhost** instance of SQL Server.
3. Use a Data Profiling task to generate the following profile requests for data in the **InternetSales** database:
 - Column statistics for the **OrderDate** column in the **SalesOrderHeader** table. You will use this data to find the earliest and latest dates on which orders have been placed.
 - Column length distribution for the **AddressLine1** column in the **Customers** table. You will use this data to determine the appropriate column length to allow for address data.
 - Column null ratio for the **AddressLine2** column in the **Customers** table. You will use this data to determine how often the second line of an address is null.
 - Value inclusion for matches between the **PaymentType** column in the **SalesOrderHeader** table and the **PaymentTypeKey** column in the **PaymentTypes** table. Do not apply an inclusion threshold and set a maximum limit of 100 violations. You will use this data to find out if any orders have payment types that are not present in the table of known payment types.
4. Run the SSIS package and view the report that the Data Profiling task generates in the Data Profile Viewer.

Results: After this exercise, you should have a comma-separated text file that contains a sample of customer data, and a data profile report that shows statistics for data in the **InternetSales** database.

Exercise 2: Transferring Data by Using a Data Flow Task

Scenario

Now you have explored the source data in the **InternetSales** database, you are ready to start implementing data flows for the ETL process. A colleague has already implemented data flows for reseller sales data, and you plan to model your Internet sales data flows on those.

The main tasks for this exercise are as follows:

1. Examine an Existing Data Flow
2. Create a Data Flow Task
3. Add a Data Source Component to a Data Flow
4. Add a Data Destination Component to a Data Flow
5. Test the Data Flow Task

► Task 1: Examine an Existing Data Flow

1. Open the **D:\Labfiles\Lab06\Starter\Ex2\AdventureWorksETL.sln** solution in SSDT.
2. Open the **Extract Reseller Data.dtsx** package and examine its control flow. Note that it contains two Data Flow tasks.
3. On the **Data Flow** tab, view the **Extract Resellers** task and note that it contains a source component named **Resellers** and a destination component named **Staging DB**.
4. Examine the **Resellers** source component, noting the connection manager that it uses, the source of the data, and the columns that its output contains.
5. Examine the **Staging DB** destination component, noting the connection manager that it uses, the destination table for the data, and the mapping of input columns to destination columns.
6. Right-click anywhere on the Data Flow design surface, click **Execute Task**, and then observe the data flow as it runs, noting the number of rows transferred.
7. When the data flow has completed, stop the debugging session.

► Task 2: Create a Data Flow Task

1. Add a new package to the project and name it **Extract Internet Sales Data.dtsx**.
2. Add a Data Flow task named **Extract Customers** to the new package's control flow.

► Task 3: Add a Data Source Component to a Data Flow

1. Create a new project-level OLE DB connection manager that uses Windows authentication to connect to the **InternetSales** database on the localhost instance of SQL Server.
2. In the **Extract Customers** data flow, add a source component that uses the connection manager you created for the **InternetSales** database, and name it **Customers**.
3. Configure the **Customers** source component to extract all columns from the **Customers** table in the **InternetSales** database.

► Task 4: Add a Data Destination Component to a Data Flow

1. Add a destination component that uses the existing **localhost.Staging** connection manager to the **Extract Customers** data flow, and then name it **Staging DB**.
2. Connect the output from the **Customers** source component to the input of the **Staging DB** destination component.
3. Configure the **Staging DB** destination component to load data into the **Customers** table in the **Staging** database.
4. Ensure that all columns are mapped, and in particular that the **CustomerKey** input column is mapped to the **CustomerBusinessKey** destination column.

► Task 5: Test the Data Flow Task

1. Right-click anywhere on the Data Flow design surface, click **Execute Task**, and then observe the data flow as it runs, noting the number of rows transferred.
2. When the data flow has completed, stop the debugging session.

Results: After this exercise, you should have an SSIS package that contains a single Data Flow task, which extracts customer records from the **InternetSales** database and inserts them into the **Staging** database.

Exercise 3: Using Transformation Components in a Data Flow

Scenario

You have implemented a simple data flow to transfer customer data to the staging database. Now you must implement a data flow for Internet sales records. The new data flow adds a new column containing the total sales amount for each line item (which is derived by multiplying the list price by the quantity of units purchased), and uses a product key value to find additional data in a separate Products database. Once again, you will model your solution on a data flow that a colleague has already implemented for reseller sales data.

The main tasks for this exercise are as follows:

1. Examine an Existing Data Flow
2. Create a Data Flow Task
3. Add a Data Source Component to a Data Flow
4. Add a Derived Column Transformation Component to a Data Flow
5. Add a Lookup Transformation Component to a Data Flow
6. Add a Data Destination Component to a Data Flow
7. Test the Data Flow Task

► Task 1: Examine an Existing Data Flow

1. Open the **D:\Labfiles\Lab06\Starter\Ex3\AdventureWorksETL.sln** solution in SSDT.
2. Open the **Extract Reseller Data.dtsx** package and examine its control flow. Note that it contains two Data Flow tasks.
3. On the **Data Flow** tab, view the **Extract Reseller Sales** task.
4. Examine the **Reseller Sales** source component, noting the connection manager that it uses, the source of the data, and the columns that its output contains.
5. Examine the **Calculate Sales Amount** transformation component, noting the expression that it uses to create a new derived column.
6. Examine the **Lookup Product Details** transformation component, noting the connection manager and query that it uses to look up product data, and the column mappings used to match data and add columns to the data flow.
7. Examine the arrows from the **Lookup Product Details** transformation component to the **Staging DB** destination component and the **Orphaned Sales** destination component.
8. Examine the **Staging DB** destination component, noting the connection manager that it uses, the destination table for the data, and the mapping of input columns to destination columns.
9. Examine the **Orphaned Sales** destination component, noting the connection manager.
10. Right-click anywhere on the Data Flow design surface, click **Execute Task**, and then observe the data flow as it runs, noting the number of rows transferred.
11. When the data flow has completed, stop the debugging session.

► **Task 2: Create a Data Flow Task**

1. Open the **Extract Internet Sales Data.dtsx** package, and then add a new Data Flow task named **Extract Internet Sales** to its control flow.
2. Connect the pre-existing **Extract Customers** Data Flow task to the new **Extract Internet Sales** task.

► **Task 3: Add a Data Source Component to a Data Flow**

1. Add a source component that uses the existing **localhost.InternetSales** connection manager to the **Extract Internet Sales** data flow, and then name it **Internet Sales**.
2. Configure the **Internet Sales** source component to use the Transact-SQL code in the **D:\Labfiles\Lab06\Starter\Ex3\InternetSales.sql** query file to extract Internet sales records.

► **Task 4: Add a Derived Column Transformation Component to a Data Flow**

1. Add a derived column transformation component named **Calculate Sales Amount** to the **Extract Internet Sales** data flow.
2. Connect the output from the **InternetSales** source component to the input of the **Calculate Sales Amount** transformation component.
3. Configure the **Calculate Sales Amount** transformation component to create a new column named **SalesAmount**, containing the **UnitPrice** column value multiplied by the **OrderQuantity** column value.

► **Task 5: Add a Lookup Transformation Component to a Data Flow**

1. Add a lookup transformation component named **Lookup Product Details** to the **Extract Internet Sales** data flow.
2. Connect the output from the **Calculate Sales Amount** transformation component to the input of the **Lookup Product Details** transformation component.
3. Configure the **Lookup Product Details** transformation component to:
 - Redirect unmatched rows to the no match output.
 - Use the **localhost.Products** connection manager and the **Products.sql** query in the **D:\Labfiles\Lab06\Starter\Ex3** folder to retrieve product data.
 - Match the **ProductKey** input column to the **ProductKey** lookup column.
 - Add all lookup columns other than **ProductKey** to the data flow.
4. Add a flat file destination component named **Orphaned Sales** to the **Extract Internet Sales** data flow. Redirect nonmatching rows from the **Lookup Product Details** transformation component to the **Orphaned Sales** destination component, which should save any orphaned records in a comma-delimited file named **Orphaned Internet Sales.csv** in the **D:\Labfiles\Lab06\Starter\Ex3** folder.

► **Task 6: Add a Data Destination Component to a Data Flow**

1. Add a destination component that uses the **localhost.Staging** connection manager to the **Extract Customers** data flow, and name it **Staging DB**.
2. Connect the match output from the **Lookup Product Details** transformation component to the input of the **Staging DB** destination component.
3. Configure the **Staging DB** destination component to load data into the **InternetSales** table in the **Staging** database. Ensure that all columns are mapped. In particular, ensure that all Key input columns are mapped to the equivalent BusinessKey destination columns.

► **Task 7: Test the Data Flow Task**

1. Right-click anywhere on the Data Flow design surface, click **Execute Task**, and then observe the data flow as it runs, noting the number of rows.
2. When the data flow has completed, stop the debugging session.

Results: After this exercise, you should have a package that contains a Data Flow task including derived column and lookup transformation components.